

Projeto Portal de Empregos - Avaliação Formativa Tarde

Projeto:

Sistema Web de Gerenciamento de Empregos e Currículo

Descrição do Projeto:

Site de vagas de emprego onde os usuários possam cadastrar seus currículos e se candidatar a vagas, enquanto as empresas possam cadastrar suas informações e publicar oportunidades de emprego.

Elaboração do Escopo e Objetivos do Projeto:

Escopo:

O projeto consiste no desenvolvimento de uma plataforma web de vagas de emprego utilizando Laravel. O sistema permitirá que candidatos criem contas, cadastrem seus currículos e se candidatem a vagas de emprego. Empresas poderão se cadastrar, criar e gerenciar vagas de emprego, bem como visualizar as candidaturas recebidas. O sistema incluirá também funcionalidades de pesquisa e filtragem de vagas, autenticação e autorização de usuários, e dashboards para administração das funcionalidades.

Objetivos:

Específicos:

- Desenvolver um sistema de cadastro e autenticação para candidatos e empresas.
- Implementar funcionalidades para candidatos visualizarem, filtrarem e se candidatarem a vagas de emprego.
- Permitir que empresas registrem e gerenciem suas vagas de emprego.
- Criar dashboards específicos para candidatos e empresas, com funcionalidades relevantes para cada perfil de usuário.
- Garantir a segurança dos dados dos usuários e a integridade das transações dentro do sistema.

Mensuráveis:

- Implementação de uma plataforma web funcional, com um mínimo de 80% de cobertura de testes unitários.
- Redução do tempo de resposta do sistema para consultas de vagas para menos de 2 segundos.
- Atingir um nível de satisfação de usuários acima de 85% em testes de usabilidade.
- Garantir que o sistema suporte até 1000 usuários simultâneos sem degradação de desempenho.

Atingíveis:

- Utilizar frameworks e bibliotecas bem documentadas (como Laravel) para assegurar o cumprimento dos prazos e qualidade do código.
- Definir entregas em ciclos de desenvolvimento ágil (sprints), com revisões e feedback constantes.
- Estabelecer uma equipe de desenvolvimento com experiência em Laravel e projetos similares.

Relevantes:

- Atender à necessidade crescente de plataformas eficientes e seguras para busca e candidatura de vagas de emprego.
- Facilitar a comunicação entre candidatos e empregadores, otimizando o processo de recrutamento.
- Prover uma solução escalável que possa ser expandida conforme o crescimento da base de usuários.

Temporais:

- Concluir o desenvolvimento do sistema em 6 meses, com entregas iterativas a cada 4 semanas.
- Implementar e testar o módulo de autenticação até o final do segundo mês.
- Finalizar o módulo de candidaturas até o quarto mês.
- Realizar testes beta e ajustes finais no último mês do cronograma.

Planejamento do Projeto:

Cronograma:

1:

- Configuração do ambiente de desenvolvimento.
- Implementação do sistema de autenticação e autorização.
- Criação das páginas de registro e login para candidatos e empresas.

2:

- Desenvolvimento do módulo de cadastro e gestão de vagas para empresas.
- Implementação da funcionalidade de pesquisa e filtragem de vagas.
- Criação do módulo de candidaturas para candidatos.

3:

- Implementação dos dashboards para candidatos e empresas.
- Testes de usabilidade e performance.
- Ajustes e melhorias com base no feedback.

4:

- Testes beta com um grupo de usuários selecionados.
- Correção de bugs e refinamento da interface.
- Preparação para lançamento e implementação de estratégias de marketing.

Recursos:

Equipe:

- 1 Gerente de Projetos
- 2 Desenvolvedores Laravel
- 1 Designer UI/UX
- 1 Especialista em Testes e QA

- 1 Analista de Marketing

Tecnologias:

- Laravel (Framework PHP)
- MySQL (Banco de Dados)
- Bootstrap (Front-end)
- AWS/Heroku (Hospedagem)
- Git/GitHub (Controle de Versão)

Ferramentas de Gestão:

- Jira/Trello para gerenciamento de tarefas
- Teams para comunicação interna
- Figma para design de interfaces

Análise de Riscos:

Risco: Atrasos no cronograma devido a requisitos não previstos.

- Mitigação: Realizar reuniões regulares com stakeholders para garantir o alinhamento das expectativas e ajustes de escopo.

Risco: Falhas de segurança que comprometem dados dos usuários.

- Mitigação: Implementar práticas de segurança desde o início do desenvolvimento e realizar auditorias de segurança regularmente.

Risco: Baixa adesão de usuários no lançamento.

- Mitigação: Desenvolver uma estratégia de marketing eficaz, incluindo campanhas de pré-lançamento para gerar interesse.

Risco: Sobrecarga no sistema com o aumento rápido de usuários.

- Mitigação: Implementar soluções escaláveis e realizar testes de carga para garantir a performance do sistema sob alta demanda.

Desenvolvimento:

Diagramas de Desenvolvimento

- Diagrama de Classe
- Diagrama de Uso
- Diagrama de Fluxo

Criação do Projeto

```
composer create-project laravel/laravel PortalVagas --prefer-dist
```

Criar o Banco de Dados no PostgreSQL

```
CREATE DATABASE portal_vagas;
```

Preparar o .env

```
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1 #localhost
DB_PORT=5432
DB_DATABASE=portal_vagas
DB_USERNAME=postgres
DB_PASSWORD=postgres
```

Rodar as Migrações do Banco de Dados(dentro do projeto)

```
php artisan migrate
```

Criar os Modelos e as Migrations(modelos de tabelas do BD)

```
php artisan make:model Usuario -m
```

```
php artisan make:model Vaga -m
```

```
php artisan make:model Inscricao -m
```

Configuração dos Modelos

Configure os modelos para refletirem os relacionamentos definidos nas migrações.

Modelo Usuario

Edite o modelo Usuario em app/Models/Usuario.php:

```
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class Usuario extends Authenticatable
{
    use Notifiable, HasFactory;
    protected $fillable = [
        'nome', 'email', 'password', 'tipo', 'cnpj', 'nome_empresa'
    ];
    protected $hidden = [
        'password', 'remember_token',
    ];
    public function inscricoes()
    {
        return $this->hasMany(Inscricao::class);
    }
}
```

Modelo Vaga

Edite o modelo Vaga em app/Models/Vaga.php:

```
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Vaga extends Model
{
    use HasFactory;

    protected $fillable = [
```

```

        'titulo', 'descricao', 'localizacao', 'salario',
        'empresa_id',
    ];

    public function empresa()
    {
        return $this->belongsTo(Empresa::class);
    }

    public function inscricoes()
    {
        return $this->hasMany(Inscricao::class);
    }
}

```

Modelo Inscricao

Edite o modelo Inscricao em app/Models/Inscricao.php:

```

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Inscricao extends Model
{
    use HasFactory;

    protected $fillable = [
        'usuario_id', 'vaga_id', 'status',
    ];

    public function usuario()
    {
        return $this->belongsTo(Usuario::class);
    }

    public function vaga()
    {

```

```

        return $this->belongsTo(Vaga::class);
    }
}

```

Definir as Migrações:

Edite o arquivo de migration criado para Usuario em
database/migrations/{timestamp}_create_usuarios_table.php:

```

public function up(): void
{
    Schema::create('usuarios', function (Blueprint $table) {
        $table->id();
        $table->string('nome');
        $table->string('email')->unique();
        $table->timestamp('email_verified_at')->nullable();
        $table->string('password');
        $table->enum('tipo', ['usuario',
            'empresa'])->default('usuario');
        $table->string('cnpj')->nullable();
        $table->string('nome_empresa')->nullable();
        $table->rememberToken();
        $table->timestamps();
    });
}

/**
 * Reverse the migrations.
 */
public function down(): void
{
    Schema::dropIfExists('usuarios');
}

```

Edite o arquivo de migration criado para Vaga em
database/migrations/{timestamp}_create_vagas_table.php:


```

public function up(): void
{
    Schema::create('vagas', function (Blueprint $table) {
        $table->id();
        $table->string('titulo');
        $table->text('descricao');
        $table->string('localizacao');
        $table->decimal('salario', 10, 2)->nullable();

        $table->foreignId('empresa_id')->constrained('empresas')->
            onDelete('cascade'); // Relaciona a vaga à empresa

        $table->timestamps();
    });
}

public function down(): void
{
    Schema::dropIfExists('vagas');
}

```

Edite o arquivo de migration criado para Inscrição em
database/migrations/{timestamp}_create_inscricoes_table.php:

```

public function up(): void
{
    Schema::create('inscricoes', function (Blueprint $table) {
        $table->id();

        $table->foreignId('usuario_id')->constrained('usuarios')->
            onDelete('cascade'); // Relaciona a inscrição ao
usuário

        $table->foreignId('vaga_id')->constrained('vagas')->
            onDelete('cascade'); // Relaciona a inscrição à vaga
        $table->string('status')->default('pendente');
        $table->timestamps();
    });
}

```

```
public function down(): void
{
    Schema::dropIfExists('inscricoes');
}
```

Rodar as Migrações

Agora que todas as migrações foram criadas, execute:

```
php artisan migrate
```

Implementar a Lógica de Autenticação no UsuarioController

Criar o UsuarioController:

```
php artisan make:controller UsuarioController
```

Adicionar Métodos de Login, Registro e Logout:

No arquivo app/Http/Controllers/UsuarioController.php, adicione os métodos para gerenciar a autenticação:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Hash;
use App\Models\Usuario;

class UsuarioController extends Controller
{
    // Exibir o formulário de login
    public function showLoginForm()
    {
        return view('usuarios.login');
    }

    // Processar o login do usuário
}
```

```

public function login(Request $request)
{
    // Validações para o login
    $credentials = $request->validate([
        'email' => ['required', 'email'],
        'password' => ['required'],
    ]);

    // Tenta autenticar com o guard 'usuario'
    if (Auth::guard('usuario')->attempt($credentials)) {
        $request->session()->regenerate(); // Regenera a sessão
para evitar fixação de sessão
        return redirect()->intended('/dashboard');
    }

    // Se falhar, retorna com erro
    return back()->withErrors([
        'email' => 'As credenciais não correspondem aos nossos
registros.',
    ])->onlyInput('email');
}

// Exibir o formulário de registro
public function showRegistroForm()
{
    return view('usuarios.registro');
}

// Processar o registro de um novo usuário
public function registro(Request $request)
{
    // Validações para o registro
    $request->validate([
        'nome' => 'required|string|max:255',
        'email' =>
'required|string|email|max:255|unique:usuarios',
        'password' => 'required|string|min:8|confirmed',
    ]);

    // Cria um novo usuário

```

```

        $usuario = Usuario::create([
            'nome' => $request->nome,
            'email' => $request->email,
            'password' => Hash::make($request->password),
        ]);

        // Faz login automático do novo usuário
        Auth::guard('usuario')->login($usuario);

        return redirect('/dashboard');
    }

    // Realizar o logout do usuário
    public function logout(Request $request)
    {
        Auth::guard('usuario')->logout(); // Logout do guard
        'usuario'
        $request->session()->regenerateToken(); // Regenera o token
        da sessão

        $request->session()->invalidate();
        $request->session()->regenerate(); // Invalida a sessão

        return redirect('/');
    }
}

```

Criar as Rotas para usuário Controller:

No arquivo routes/web.php, defina as rotas para login, registro e logout utilizando o UsuarioController:

```

<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\UsuarioController;

// Rota para exibir a homePage
Route::get('/', function() {return view('home');});

```

```

// Rota para exibir o formulário de login
Route::get('/login', [UsuarioController::class, 'showLoginForm'])->
name('usuarios.login');

// Rota para processar o login
Route::post('/login', [UsuarioController::class, 'login'])->
name('usuarios.login');

// Rota para exibir o formulário de registro
Route::get('/registro', [UsuarioController::class,
'showRegisterForm'])->
name('usuarios.register');

// Rota para processar o registro
Route::post('/registro', [UsuarioController::class, 'register'])->
name('usuarios.register');

// Rota para logout
Route::post('/logout', [UsuarioController::class, 'logout'])->
name('usuarios.logout');

// Rota para o dashboard, protegida por autenticação
Route::get('/dashboard', function () {
    return view('dashboard');
})->middleware('auth')->name('dashboard');

```

Prepara as View do Projeto

Crie o Arquivo layouts.app.blade.php

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{{ config('app.name', 'Portal Vagas') }}</title>

```

```

    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.
css" rel="stylesheet">
</head>
<body>

    @include('parts.header')

    <div class="container">
        @yield('content')
    </div>

    @include('parts.footer')

</body>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle
.min.js"></script>
</html>

```

Criar a home.blade.php

Criar a usuarios.registro.blade.php

```

@extends('layouts.app')

@section('content')
    {{-- formulario --}}
    <div class="container">
        <h1>Registrar-se</h1>
        <form method="POST" action="{{ route('usuarios.registro') }}">
            @csrf

            <div class="form-group">
                <label for="name">Nome</label>
                <input type="text" name="name" class="form-control" required>
            </div>

```

```

        <div class="form-group">
            <label for="email">Email</label>
            <input type="email" name="email" class="form-control"
required>
        </div>

        <div class="form-group">
            <label for="password">Senha</label>
            <input type="password" name="password" class="form-control"
required>
        </div>

        <div class="form-group">
            <label for="password_confirmation">Confirme a Senha</label>
            <input type="password" name="password_confirmation"
class="form-control" required>
        </div>

        <button type="submit" class="btn
btn-primary">Registrar-se</button>
    </form>
</div>

```

@endsection

Criar a Página usuarios.login.blade.php

```

@extends('layouts.app')

@section('content')
<div class="container">
    <h1>Login</h1>
    <form method="POST" action="{{ route('usuarios.login') }}">
        @csrf

        <div class="form-group">
            <label for="email">Email</label>
            <input type="email" name="email" class="form-control" required
autofocus>

```

```

        </div>

        <div class="form-group">
            <label for="password">Senha</label>
            <input type="password" name="password" class="form-control"
required>
        </div>

        <button type="submit" class="btn btn-primary">Login</button>
    </form>
</div>
@endsection

```

<https://pt.stackoverflow.com/questions/279238/usu%C3%A1rio-n%C3%A3o-fica-autenticado-autom%C3%A1tico>

config/auth.php editar para:

```

<?php

return [

    /*
    |-----
    |
    | Authentication Defaults
    |-----
    |
    | This option defines the default authentication "guard" and password
    | reset "broker" for your application. You may change these values
    | as required, but they're a perfect start for most applications.
    |
    */

    'defaults' => [

```



```

        'guard' => 'usuario',
        'passwords' => 'users',
    ],

    /*
|-----
-
| Authentication Guards
|
|-----
-
|
| Next, you may define every authentication guard for your
application.
| Of course, a great default configuration has been defined for you
| which utilizes session storage plus the Eloquent user provider.
|
| All authentication guards have a user provider, which defines how
the
| users are actually retrieved out of your database or other storage
| system used by the application. Typically, Eloquent is utilized.
|
| Supported: "session"
|
*/

    'guards' => [
        'usuario' => [
            'driver' => 'session',
            'provider' => 'usuario',
        ],
        'api' => [
            'driver' => 'token',
            'provider' => 'users',
        ],
    ],
    /*

```

```

|-----
-
    | User Providers

|-----
-
    |
    | All authentication guards have a user provider, which defines how
the
    | users are actually retrieved out of your database or other storage
    | system used by the application. Typically, Eloquent is utilized.
    |
    | If you have multiple user tables or models you may configure
multiple
    | providers to represent the model / table. These providers may then
    | be assigned to any extra authentication guards you have defined.
    |
    | Supported: "database", "eloquent"
    |
    */

    'providers' => [
        'usuario' => [
            'driver' => 'eloquent',
            'model' => App\Models\Usuario::class,
        ],

        // 'users' => [
        //     'driver' => 'database',
        //     'table' => 'users',
        // ],
    ],

    /*

|-----
-
    | Resetting Passwords

```

```
|-----  
-  
    |  
    | These configuration options specify the behavior of Laravel's  
password  
    | reset functionality, including the table utilized for token storage  
    | and the user provider that is invoked to actually retrieve users.  
    |  
    | The expiry time is the number of minutes that each reset token will  
be  
    | considered valid. This security feature keeps tokens short-lived so  
    | they have less time to be guessed. You may change this as needed.  
    |  
    | The throttle setting is the number of seconds a user must wait  
before  
    | generating more password reset tokens. This prevents the user from  
    | quickly generating a very large amount of password reset tokens.  
    |  
    */
```

```
'passwords' => [  
    'users' => [  
        'provider' => 'users',  
        'table' => 'password_resets',  
        'expire' => 60,  
    ],  
],  
  
/*
```

```
|-----  
-  
    | Password Confirmation Timeout
```

```
|-----  
-  
    |  
    | Here you may define the amount of seconds before a password  
confirmation
```

```

        | window expires and users are asked to re-enter their password via
the
        | confirmation screen. By default, the timeout lasts for three hours.
        |
        */

        'password_timeout' => env('AUTH_PASSWORD_TIMEOUT', 10800),

];

```

Criar uma NavBar de navegação Modificada conforme a etapa do usuário

views/parts/header.blade.php

```

@if (Auth::check())
    <div>
        <h3>Olá, {{Auth::user()->nome}}</h3>
    </div>
    <div>
        <form action="/logout" method="post">
            @csrf
            <input type='submit' value='Sair'>
        </form>
    </div>
    <br>
    <hr>
    <br>
@else
    <div class="nav-bar">
        <a href="/login">Login</a>
        <a href="/registro">Registre-se</a>
    </div>
    <br>
    <hr>
    <br>
@endif

```

Ajustar o Model Vaga e o Create_Vagas_Table

Criar o VagaController

```
php artisan make:controller VagaController --resource
```

Vai Criar o arquivo

App\Http\Controller\VagaController.php

```
<?php
```

```
namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\Vaga;

class VagaController extends Controller
{
    /**
     * Display a listing of the resource.
     */
    public function index()
    {
        $vagas = Vaga::all();
        return view('vagas.index', compact('vagas'));
    }

    /**
     * Show the form for creating a new resource.
     */
    public function create()
    {
        return view('vagas.create');
    }

    /**
     * Store a newly created resource in storage.
     */
    public function store(Request $request)
    {

```

```

        $dados = $request->validate([
            'titulo'=> 'required|max:100',
            'descricao'=> 'required',
            'localizacao'=> 'required',
            'salario'=>'required|numeric',
            'empresa'=>'required'
        ]);
        Vaga::create($dados);

        return redirect()->route('vagas.index')
            ->with('success', 'Vaga criado com sucesso.');
```

}

```

/**
 * Show the form for editing the specified resource.
 */
public function edit(Vaga $vaga)
{
    return view('vagas.edit',compact('vaga'));
}

/**
 * Update the specified resource in storage.
 */
public function update(Request $request, Vaga $vaga)
{
    $dados = $request->validate([
        'titulo'=> 'required|max:100',
        'descricao'=> 'required',
        'localizacao'=> 'required',
        'salario'=>'required|numeric',
        'empresa'=>'required'
    ]);
    $vaga->update($dados);

    return redirect()->route('vagas.index')
        ->with('success', 'Vaga Atualizada com sucesso.');
```

}

```

/**
 * Remove the specified resource from storage.
 */
public function destroy(Vaga $vaga)
{
    $vaga->delete($vaga);

    return redirect()->route('vagas.index')
        ->with('success', 'Vaga Deletada com sucesso.');
```

Criar as View para Vagas

views/vagas/index.blade.php - php artisan make:view vagas.index

```

@extends('layouts.app')

@section('content')
    <div class="container">
        <h1 class="my-4">Vagas</h1>

        @if ($message = Session::get('success'))
            <div class="alert alert-success">
                <p>{{ $message }}</p>
            </div>
        @endif

        <a class="btn btn-success mb-2" href="{{ route('vagas.create') }}"> Criar Nova Vaga</a>

        <table class="table table-bordered">
            <tr>
                <th>Nº</th>
                <th>Título</th>
                <th>Descrição</th>
                <th>Localização</th>
```

```

        <th>Salário</th>
        <th>Empresa</th>
        <th width="280px">Ação</th>
    </tr>
    @foreach ($vagas as $vaga)
    <tr>
        <td>{{ $loop->iteration }}</td>
        <td>{{ $vaga->titulo }}</td>
        <td>{{ $vaga->descricao }}</td>
        <td>{{ $vaga->localizacao }}</td>
        <td>{{ $vaga->salario }}</td>
        <td>{{ $vaga->empresa }}</td>
        <td>
            <form action="{{ route('vagas.destroy', $vaga->id) }}"
method="POST">
                <a class="btn btn-primary" href="{{
route('vagas.edit', $vaga->id) }}">Editar</a>
                @csrf
                @method('DELETE')
                <button type="submit" class="btn
btn-danger">Deletar</button>
            </form>
        </td>
    </tr>
    @endforeach
</table>
</div>
@endsection

```

views/vagas/create.blade.php - php artisan make:view vagas.create

```

@extends('layouts.app')

@section('content')
<div class="container">
    <h1 class="my-4">Criar Vaga</h1>

    @if ($errors->any())

```



```
<div class="alert alert-danger">
    <strong>Oops!</strong> Houve alguns problemas com sua
entrada.<br><br>
    <ul>
        @foreach ($errors->all() as $error)
            <li>{{ $error }}</li>
        @endforeach
    </ul>
</div>
@endif

<form action="{{ route('vagas.store') }}" method="POST">
    @csrf
    <div class="form-group">
        <label for="nome">Título:</label>
        <input type="text" name="titulo" class="form-control"
placeholder="Título">
    </div>

    <div class="form-group">
        <label for="descricao">Descrição:</label>
        <textarea name="descricao" class="form-control"
placeholder="Descrição"></textarea>
    </div>

    <div class="form-group">
        <label for="localizacao">Localização:</label>
        <input type="text" name="localizacao" class="form-control"
placeholder="Localização">
    </div>

    <div class="form-group">
        <label for="salario">Salário:</label>
        <input type="text" name="salario" class="form-control"
placeholder="Salário">
    </div>

    <div class="form-group">
        <label for="empresa">Empresa:</label>
```

```

        <input type="text" name="empresa" class="form-control"
placeholder="Empresa">
    </div>

    <button type="submit" class="btn btn-primary">Enviar</button>
</form>
</div>
@endsection

```

views/vagas/edit.blade.php - php artisan make:view vagas.edit

```

@extends('layouts.app')

@section('content')
<div class="container">
    <h1 class="my-4">Criar Vaga</h1>

    @if ($errors->any())
        <div class="alert alert-danger">
            <strong>Oops!</strong> Houve alguns problemas com sua
entrada.<br><br>
            <ul>
                @foreach ($errors->all() as $error)
                    <li>{{ $error }}</li>
                @endforeach
            </ul>
        </div>
    @endif

    <form action="{{ route('vagas.update', $vaga->id) }}" method="POST">
        @csrf
        @method('PUT')
        <div class="form-group">
            <label for="nome">Título:</label>
            <input type="text" name="titulo" class="form-control"
value="{{ $vaga->titulo }}">
        </div>

        <div class="form-group">

```

```

        <label for="descricao">Descrição:</label>
        <input type="text" name="descricao" class="form-control"
value="{{ $vaga->descricao }}">
    </div>

    <div class="form-group">
        <label for="localizacao">Localização:</label>
        <input type="text" name="localizacao" class="form-control"
value="{{ $vaga->localizacao }}">
    </div>

    <div class="form-group">
        <label for="salario">Salário:</label>
        <input type="text" name="salario" class="form-control"
value="{{ $vaga->salario }}">
    </div>

    <div class="form-group">
        <label for="empresa">Empresa:</label>
        <input type="text" name="empresa" class="form-control"
value="{{ $vaga->empresa }}">
    </div>

    <button type="submit" class="btn btn-primary">Enviar</button>
</form>
</div>
@endsection

```

Criar a Restrição de acesso Via Middleware

php artisan make:middleware VagaMiddleware

Cria a class App/Http/Middleware/VagaMiddleware.php

```

<?php

namespace App\Http\Middleware;

use Closure;

```

```

use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;
use Illuminate\Support\Facades\Auth;

class VagaMiddleware
{
    /**
     * Handle an incoming request.
     *
     * @param  \Closure(\Illuminate\Http\Request): (\Symfony\Component\HttpFoundation\Response)  $next
     */
    public function handle(Request $request, Closure $next): Response
    {
        if(Auth::check() && Auth::user()->tipo === 'empresa'){
            return $next($request);
        }

        // Se não for uma empresa, redireciona com uma mensagem de erro
        return redirect()->route('')->
            withErrors(['access' => 'Você não tem permissão para acessar
            essa área.']);
    }
}

```

Vamos Criar a Rota

```

Route::resource('/vagas',VagaController::class)->
    middleware(VagaMiddleware::class);

```

views.vagas.index -

Ajustando para exibir vagas somente da empresa do usuário empresa

Editando o vagasController.php - index

```

public function index()
{
    $empresa = Auth::user()->nome_empresa;
    $vagas = Vaga::where('empresa',$empresa)->get();
}

```

```

        return view('vagas.index', compact('vagas'));
    }

```

Criando a HomeController -

Código para views.home.blade.php

```

@extends('layouts.app')

@section('content')
    <div class="container">
        <div id="productCarousel" class="carousel slide"
data-bs-ride="carousel">
            <div class="carousel-inner">
                @foreach ($vagas as $index => $vaga)
                    <div class="carousel-item {{ $index == 0 ? 'active' :
'' }}">

                        titulo }}">
                        <div class="carousel-caption d-none d-md-block">
                            <h5>{{ $vaga->titulo }}</h5>
                            <h5>{{ $vaga->empresa }}</h5>
                            <p>{{ $vaga->descricao }}</p>
                            <p>Salário: R$ {{ $vaga->salario
}}</p>

                        </div>
                    </div>
                @endforeach
            </div>
            <button class="carousel-control-prev" type="button"
data-bs-target="#productCarousel" data-bs-slide="prev">
                <span class="carousel-control-prev-icon"
aria-hidden="true"></span>
                <span class="visually-hidden">Previous</span>
            </button>

```

```

        <button class="carousel-control-next" type="button"
data-bs-target="#productCarousel" data-bs-slide="next">
            <span class="carousel-control-next-icon"
aria-hidden="true"></span>
            <span class="visually-hidden">Next</span>
        </button>
    </div>
</div>
@endsection

```

Criando o Dashboard

views.usuario.dashboard.blade.php

```

@extends('layouts.app')

@section('content')
    <h1>Dashboard de Vagas</h1>

    <form method="GET" action="{{ route('dashboard') }}">
        <input type="text" name="search" placeholder="Pesquisar vagas..."
value="{{ request('search') }}">
        <button type="submit">Pesquisar</button>
    </form>

    <div class="row">
        @foreach ($vagas as $vaga)
            <div class="col-md-4">
                <div class="card">
                    titulo }}">
                    <div class="card-body">
                        <h5 class="card-title">{{ $vaga->titulo }}</h5>
                        <h5 class="card-title">{{ $vaga->empresa }}</h5>
                        <p class="card-text">{{ $vaga->descricao }}</p>

```

```

        <p class="card-text">Preço: R$ {{ $vaga->salario
    }}</p>

        <a href="{{ route('vagas.show', $vaga->id) }}"
class="btn btn-primary">Ver Vaga</a>
    </div>
</div>
</div>
@endforeach
</div>
@endsection

```

Criar o DashboardController

```
php artisan make:controller DashboardController
```

Código Para DashboardController

Criar o views.vagas.show

```

@extends('layouts.app')

@section('content')
    <div class="container">
        <div class="row">
            <div class="col-md-6">
                titulo }}">
            </div>
            <div class="col-md-6">
                <h2>{{ $vaga->titulo }}</h2>
                <h2>{{ $vaga->empresa }}</h2>
                <p>{{ $vaga->descricao }}</p>
                <p>Preço: R$ {{ $vaga->salario }}</p>

                <form method="POST" action="{{ route('pedidos.add',
$vaga->id) }}">

                    @csrf

```

```

        <button type="submit" class="btn btn-primary">Fazer
Inscrição</button>
    </form>
</div>
</div>
</div>
@endsection
```